

Container Manager (Dashboard de Operações)

1. Visão Geral

O desafio deste projeto é desenvolver uma aplicação desktop profissional para a gestão e monitoramento de containers. Em vez de reconstruir o motor do Docker, o grupo criará um **Orquestrador de Interface (Dashboard)** que se comunica diretamente com a *Docker Engine*. O sistema permitirá que administradores de sistemas visualizem o status de suas imagens e containers, executem comandos de ciclo de vida (Iniciar/Parar) e monitorem o consumo de recursos de hardware em tempo real, centralizando a operação em uma interface amigável.

2. Objetivos de Aprendizado e Aplicação (POO)

Este projeto foca na integração entre sistemas e na organização de dados complexos, utilizando os conceitos de POO:

- **Herança e Modelagem:** Implementar uma hierarquia de classes clara onde `Container`, `Image` e `Network` herdam propriedades de uma classe base de ativos do sistema, garantindo a reutilização de código e organização lógica.
- **Encapsulamento:** Proteger as informações de estado e IDs dos containers, permitindo alterações apenas através de métodos validados (ex: um container não pode ser "parado" se já estiver com o status `exited`).
- **Membros Estáticos e Finais:** Definir constantes para URLs de conexão com o daemon do Docker e limites de alertas para consumo de CPU/Memória.

3. Requisitos de Persistência (Banco de Dados)

A aplicação deve atuar como uma ferramenta de auditoria e histórico para a equipe de TI:

- **Persistence Expert (DAO):** Implementar um sistema de log persistente. Toda vez que um usuário iniciar ou parar um container, a ação deve ser registrada no banco de dados com timestamp, ID do container e o resultado da operação.
- **Logs de Erro:** Capturar exceções de conexão com a API do Docker e armazená-las no banco para diagnóstico futuro, utilizando blocos `try-catch` robustos.

4. Bônus: Interface Gráfica

O Dashboard deve ser funcional e visualmente informativo:

- **Monitoramento em Tempo Real:** Uso de `ProgressBar` ou `Charts` para exibir o consumo de CPU e RAM dos containers ativos.
- **Organização de Layout:** Utilizar `GridPane` para criar um painel modular, onde cada container é representado por um "card" com informações de status e botões de ação.
- **Controle de Eventos:** Vincular os botões da interface (Play, Stop, Refresh) aos comandos enviados à API, garantindo que a tela seja atualizada automaticamente após a execução.

5. Responsabilidades da Equipe (Equipe de 3 Alunos)

Para garantir a eficiência em 8 semanas, o grupo deve se organizar nos seguintes papéis:

1. **API Integrator:** Gerencia a comunicação entre o Java e o Docker (usando bibliotecas como *docker-java* ou chamadas de sistema).
2. **Model & Logic:** Desenvolve a arquitetura de classes POO e a lógica de processamento dos dados recebidos.
3. **Persistence Expert:** Cria o esquema SQL e as classes DAO para o histórico de operações.
4. **UI Designer (JavaFX):** Projeta as telas FXML e a estética do dashboard.
5. **Events Controller:** Implementa a lógica dos controladores, vinculando a UI às ações da API e persistência.

6. Entregas Esperadas

Ao final o grupo deverá apresentar:

1. **Dashboard Operacional:** Aplicação listando os containers reais da máquina e permitindo o controle deles via interface.
2. **Relatório de Performance:** Demonstração dos gráficos de consumo de recursos variando conforme o uso do container.
3. **Histórico de Banco de Dados:** Apresentação da tabela de logs contendo todas as ações realizadas durante a demonstração.